

Le Buzzer

Kahoot, sauf que cette fois c'est vous qui l'écrivez.

Introduction

Une société de production cherche un outil pour animer ses quiz en direct, en plateau comme en séminaire d'entreprise. Le principe est celui du buzzer de télévision : un animateur ouvre une partie, les joueurs la rejoignent depuis leur téléphone avec un code à cinq lettres, l'animateur envoie les questions une par une, les réponses remontent, et le classement se met à jour chez tout le monde en même temps.

Le produit est volontairement petit. Il n'y a pas de compte utilisateur, pas d'historique, pas de statistiques, et une partie peut parfaitement vivre en mémoire le temps qu'elle dure. Ce dépouillement est le sujet lui-même : **toute la semaine porte sur le temps réel**, pas sur du CRUD que vous savez déjà écrire.

Ce qui fait la difficulté d'une application temps réel, ce n'est jamais le cas nominal. C'est ce qui se passe quand un joueur arrive en retard, quand le wifi de la salle coupe trois secondes, quand deux parties tournent en parallèle, ou quand quelqu'un ferme son onglet en plein milieu. Ces cas sont annoncés dès maintenant, et ce sont eux qui sont évalués.



Le produit attendu

Les deux écrans

L'écran animateur

- créer une partie, ce qui produit un code à cinq lettres ;
- voir en direct la liste des joueurs qui rejoignent ;
- lancer une question, avec ses propositions de réponse et sa durée ;
- voir la progression des réponses pendant le temps imparti ;
- clore la question et publier le classement ;
- passer à la suivante, ou terminer la partie.

L'écran joueur

- rejoindre une partie avec un code et un pseudonyme ;
- attendre la question suivante ;
- répondre, une seule fois, dans le temps imparti ;
- voir le résultat de la question et sa position au classement.

Le questionnaire

Il est fourni par le binôme sous forme de données du projet. Une dizaine de questions suffit, le contenu n'est pas évalué. Ne perdez pas de temps à construire un éditeur de questionnaire, ce n'est pas le sujet.

Rappel : **le code s'écrit en anglais**, y compris les noms de messages, de topics et d'énumérations. Le français reste pour l'interface.



Les quatre difficultés réelles

Elles sont annoncées d'avance. Il n'y a aucun piège caché dans ce sujet, seulement des cas que les applications temps réel amateurs oublient toutes.

1. Le joueur qui rejoint en cours de partie

Un joueur qui arrive après le début doit recevoir **l'état courant**, pas seulement les événements suivants. Il doit voir la question en cours s'il y en a une, le temps restant, et le classement actuel.

C'est la différence entre un flux d'événements et un état partagé, et c'est le point que la plupart des implémentations ratent.

2. La reconnexion

La connexion peut tomber. Votre client doit :

- détecter la perte ;
- afficher un état visible pour l'utilisateur, connecté ou déconnecté ;
- retenter avec un **backoff exponentiel**, pas en boucle serrée toutes les cinquante millisecondes ;
- et surtout, **rattraper l'état** une fois reconnecté.

Un écran qui reste figé sur un état obsolète après une reconnexion est considéré comme non fonctionnel.

3. L'autorité sur le temps

Chaque question a une durée. La question à trancher, et à défendre à l'oral : **qui fait autorité sur le décompte**, le serveur ou le navigateur ?

Les deux approches ont des conséquences différentes sur la triche, sur la synchronisation entre joueurs et sur le comportement en cas de latence. Vous choisissez, et vous justifiez.



4. Les cas limites de joueur

- Un joueur qui répond deux fois à la même question : seule la première réponse compte.
- Un joueur qui ferme son onglet : il disparaît de la liste des connectés sans faire tomber la partie.
- Deux parties qui tournent en même temps : un message destiné à la partie A ne doit **jamais** arriver dans la partie.

Contraintes techniques

Backend

- Spring Boot 3 avec WebSocket et STOMP.
- **Diffusion ciblée** : les messages d'une partie vont aux membres de cette partie, pas à tous les clients connectés. C'est la notion de salle, et elle est évaluée.
- La logique de partie vit dans la **couche service**. Un controller WebSocket reçoit, délègue, publie. Comme en REST, et pour les mêmes raisons.
- La persistance n'est pas exigée. Une partie peut vivre en mémoire.

Frontend

- React 18, TypeScript strict, strict: true, aucun any.
- Projet créé avec Vite.
- La logique de connexion vit dans un **hook personnalisé ou un service dédié**, jamais éparpillée dans les composants.
- Le désabonnement au démontage du composant est obligatoire, et vous devez savoir dire ce qui se passe quand on l'oublie.



L'étude comparative

Vous produisez une note de deux pages maximum comparant, sur ce cas précis, le polling HTTP et les WebSockets.

Elle doit être chiffrée. Le calcul de référence à poser : une partie de 10 minutes, 15 joueurs connectés, un polling toutes les 2 secondes.

- Combien de requêtes HTTP cela représente-t-il au total ?
- Combien de messages WebSocket votre implémentation échange-t-elle réellement sur la même partie ? Instrumentez et comptez, ne devinez pas.
- Quelle est la latence perçue dans chaque cas entre l'action de l'animateur et l'affichage chez le joueur ?

Concluez, et mentionnez au moins un cas où le polling reste le bon choix. Le WebSocket n'est pas supérieur en tout.

Résumé

1. Backend Spring avec WebSocket et STOMP, gestion de salles et diffusion ciblée.
2. Écran animateur et écran joueur fonctionnels, en React et TypeScript strict.
3. Gestion complète des états de connexion, avec reconnexion et rattrapage d'état.
4. Note comparative polling contre WebSocket, 2 pages maximum, chiffrée et instrumentée.
5. README avec instructions de lancement.



Compétences visées

- **BC01-C2** : développer des interfaces utilisateur
- **BC01-C3** : développer des composants métier
- **BC02-C2** : définir l'architecture logicielle d'une application

Rendu

Le projet est à rendre par binôme sur
<https://github.com/prenom-nom/LeBuzzer>

Structure attendue :

LeBuzzer/
 backend/ (projet Spring Boot)
 frontend/ (projet Vite React TS)
 docs/ (note comparative)
 [README.md](#)

Base de connaissances

Backend

- [Spring, support WebSocket](#)
- [Spring, STOMP sur WebSocket](#)
- [Spring Boot](#)

Protocole

- [MDN, API WebSocket](#)
- [MDN, écrire des applications WebSocket](#)
- [Spécification STOMP 1.2](#)

Frontend



- [React](#)
- [React, synchroniser avec des systèmes externes](#)
- [TypeScript Handbook](#)
- [Vite](#)